

VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE



NHẬP MÔN DỮ LIỆU LỚN

INTRODUCTION TO BIG DATA ANALYSIS

Lecturers

Dr. Nguyễn Ngọc Thảo | nnthao@fit.hcmus.edu.vn

Dr. Lê Ngọc Thành | lnthanh@fit.hcmus.edu.vn

Lab instructor

Huỳnh Lâm Hải Đăng | hlhdang@fit.hcmus.edu.vn

Lab 02: Advanced MapReduce & Spark Structured APIs¹

1. Preliminaries

- To do this lab, you need to know the basic concepts of Spark and Structured APIs. You can read the basics in this article ([Link](#)) or the official tutorial from Apache ([Link](#) – written in Python but it is similar in other languages).

1.1. The history of Spark

- In the early days of Spark, data operations were performed directly on RDDs, which were much harder to write and optimize.
- With the advent of DataFrame and Structured APIs, writing Spark jobs has become much easier and more efficient thanks to automatic optimization features.
- To fully and accurately understand all the concepts that power Spark like Catalyst optimizer, Whole-stage Code Gen, etc. I recommend you to read more in the book named ***Spark: The Definitive Guide***. (it is not required, but you should take a look).
- Later, more and more features were introduced, such as Adaptive Query Execution (AQE) introduced in version 3.0 to optimize Spark jobs, making them run significantly faster *in most cases*.

1.2. DataFrame

DataFrame in Spark has the following properties:

- **Schema-based**: Dataframes have schemas, which require declaring the names and data types of the columns in the dataframe (that's why they are named Structured APIs). Supports many primitive data types as well as other complex data types.
- **Immutable**: Dataframes in Spark cannot be modified after being created. However, we can transform them into another dataframe through **transformations**.
- **Lazy evaluation**: When the user has not performed an **action**, the previous transformations (in the same stage) have not been run. Once an action is triggered, the Catalyst optimizer will review the execution plan, then optimize the entire stage, and then perform processing on the optimized plan.

1.3. The underlying of Spark vs. MapReduce

Spark DataFrames and MapReduce both enable distributed data processing but differ significantly in execution and efficiency. Both frameworks process large-scale

¹ Credit: this labwork's statement and problems are also co-authored by Vũ Công Thành.

datasets in parallel across clusters, breaking computations into smaller tasks that run on multiple nodes.

However, MapReduce follows a disk-based, batch processing model, where intermediate results are written to disk between the map and reduce phases, making it slower due to frequent I/O operations. In contrast, Spark DataFrames leverage in-memory processing, significantly reducing disk I/O and improving speed.

Another key difference is that MapReduce requires users to explicitly define map and reduce functions, whereas DataFrames provide a higher-level, SQL-like API that abstracts these details, allowing for more readable and concise code.

Additionally, Spark DataFrames benefit from Catalyst Optimizer and Tungsten Engine, which apply automatic query optimizations and memory-efficient execution, whereas MapReduce lacks built-in query optimization and relies solely on user-defined logic.

Despite these differences, both frameworks perform similar fundamental operations, such as mapping, shuffling, and reducing data, but Spark DataFrames streamline this process with optimized execution plans and a more user-friendly API, making them a more efficient and flexible alternative to MapReduce.

2. Homework

In all exercises below, you can choose your preferred programming language to work with, among Java, Scala, and Python. Note that you are only allowed to fall back on the native code or libraries of that language if Apache Spark's built-in APIs are not sufficient to accomplish the required objectives. For example, Numpy is considered a native library if Python is your selected programming language.

In both exercises 2.1 and 2.2 below, you will use the same Amazon Sale Report² dataset that is included in this Lab.

2.1. Calculate revenue in the last 3 days for each country

Using the **MapReduce framework**:

1. Implement a sliding window computation to calculate the *total sales (Amount) for each Category in the last 3 days*.
2. The window should slide by 1 day at a time.
3. Sort the results in ascending order by `report_date`, followed by category.

The expected output should look like this (make sure the format is correct, column names are ALL in lowercase):

report_date	category	revenue
30/04/2022	Set	3829.38
30/04/2022	Western Dress	4392.85

² From [E-Commerce Sales Dataset](#), formatted as `asr.csv`

30/04/2022	kurta	3293.19
...	...	...

2.2. Calculate the number of products sold in the last 7 days of each SKU, report every Monday

Using **Apache Spark**:

1. Implement a computation to calculate the *total quantity (Qty) of items shipped for each SKU in the last 7 days, but only report this total every Monday.*
2. The goal is to provide a weekly snapshot of the total quantity shipped for each SKU over the past week.
3. Sort the results in ascending order by report_date, followed by SKU.

Expected output:

report_date	sku	total_quantity
02/05/2022	Set	338
02/05/2022	Western Dress	329
02/05/2022	kurta	392
...	...	...

2.3. Find pairs of overlapping shapes

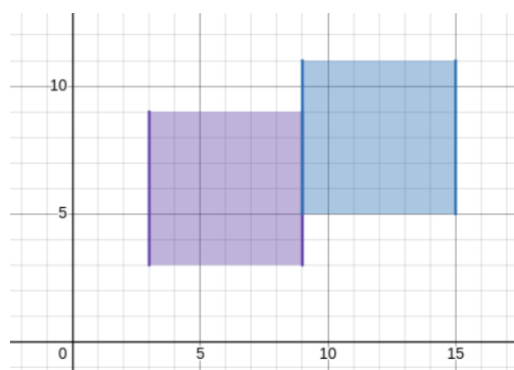
You are provided with a dataset (*shapes.parquet*) containing 1000 geometric shapes, where each shape is represented as a quadrilateral with 4 vertices. Each shape is defined by its **unique ID** and **four vertices** representing a quadrilateral in a 2D coordinate system.

Using **Apache Spark**:

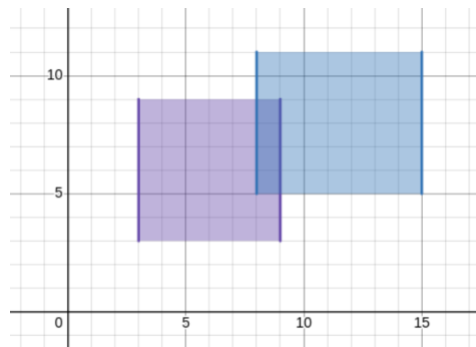
1. Implement the Overlap Detection Algorithm.
2. Output the overlapping shape pairs (their ids).
3. Sort the results in ascending order by shape_1, followed by shape_2.

Note that shapes only share points on the boundaries are NOT counted as overlapping shapes.

For example, these two shapes are **not** overlapped:



These two shapes are overlapped:



Expected output:

shape_1	shape_2
1	3
1	7
8	10
...	...

3. Submission Guideline

This lab requires a group's submission where the work of your group's members is compressed into a single file and only one representative may submit this file on Moodle. The submission file contains a single folder named *<GroupID>* where student ID of the first member (that your group has registered in earlier form) is used. This folder's internal structure contains the group's labwork for each problem. Specifically the structure will be as follow:

```

<GroupID>
|--- src
|   |--- Task_2.1
|   |--- | <GroupID>.py/<GroupID>.jar/<GroupID>.ipynb
|   |--- | output.csv
|   |--- | ...
|   |--- Task_2.2
|   |--- | <GroupID>.py/<GroupID>.jar/<GroupID>.ipynb
|   |--- | output.csv
|   |--- | ...
|   |--- Task_2.3
|   |--- | <GroupID>.py/<GroupID>.jar/<GroupID>.ipynb
|   |--- | output.csv
|   |--- README (optional, instructions to run the code)
|   |--- | ...
|--- docs
|   |--- Report.pdf

```

You must strictly follow the above file structure and compress the whole folder into a ZIP file named *<FirstMemberID>.zip*, which is your final file to be submitted to Moodle.

Grading Criteria

Requirements	Points
Exercise 2.1	3.5
- Correct Map phase	0.75
- Correct Reduce phase	0.75
- All output rows are correct	1.75
- Results are sorted in the right order	0.25
Exercise 2.2	2
- All output rows are correct	1.75
- Results are sorted in the right order	0.25
Exercise 2.3	3.5
- The overlap detection function works	0.75
- All output rows are correct	2.5
- Results are sorted in the right order	0.25
Report	1.25
- Detailed description of each problem's solution	0.5
- Reasoning behind each of such solutions	0.5
- Contribution: what task performed by which member?	0.25
TOTAL	10.25

You must strictly follow the aforementioned file structure and compress the whole folder into a ZIP file named *<GroupID>.zip*, which is your final file to be submitted to Moodle.

- Ensure your code is well-documented with clear comments.

Happy Coding and Best of Luck!

The Lab Instructor./.